BUILDING DETECTION USING U-NET MODEL

PROJECT REPORT

Department of Computer Science
Aarhus University

Department of Computer Science
Aarhus University

December 4, 2025

ABSTRACT

This project addresses the task of building segmentation on the pre-disaster subset of the xBD dataset using a U-Net with a ResNet-34 encoder. Throughout a series of experiments, we investigated factors that limited the model's ability to produce accurate building masks, including data imbalance, boundary precision, and the model's reliance on contextual cues. We evaluated several modifications—learning-rate tuning, normalization, boundary-aware loss, Grad-CAM analysis, data augmentation, and hard negative mining—but found that these had little effect compared to the choice of sampling strategy. The main performance gains came from rebalancing the training data through building-centered random crops, which exposed the model to more informative examples and improved the building IoU from 0.57 to 0.67. Our results highlight that, for this dataset, data selection plays a more decisive role than architectural or loss-function adjustments, and that controlling what the model sees is essential for achieving reliable segmentation performance.

1 Introduction

This project aims to handle the first part of the xView2 challenge xVi, meaning we aim to perform image segmentation of buildings from satellite images. This task is considered to be the first building block of the challenge, whose purpose is to have a deep learning pipeline that will detect the houses affected by natural disasters given the satellite images, in order for the very much needed special rescue teams to be able to quickly and correctly identify the impacted area. Of course, in order to be able to see if the land from a given satellite image needs aid, we first need to be able to determine what stood on that land before the natural disaster occured; because it is a big diference if there was a building, a house or if the field was bare beforehand.
Our project uses the U-Net architecture and handles all the bottlenecks found in it's path. For the final model used, we have the pre-trained encoder which is a Resnet-34 pretrained on imagenet paired with a decoder that is fine-tuned on the dataset (xBD). To be able to use the images, there was computed a binary mask for each image, applied a random crop, found a fitting learning rate and trained for multiple epochs. The results we have obtained are around 0.97 IoU for the background and 0.67 for the buildings.

2 State of the Art

Building footprint extraction from high-resolution satellite imagery is a long-standing problem in remote sensing, with recent progress driven by convolutional neural networks (CNNs) and encoder–decoder architectures. U-Net and its variants remain the dominant baseline for pixel-wise segmentation tasks Ronneberger et al. [2015], but numerous studies show that naive U-Net models struggle with boundary precision and false positives caused by vegetation, shadows, roads, and clutter around buildings. To address these issues, state-of-the-art methods incorporate stronger encoders such as ResNet-50 Alsabhan and Alotaiby [2022]. Loss-function design also plays a central role: Dice and Lovász losses

*

directly optimize region-overlap metrics Ataş [2023], while boundary-aware weighting improves edge localization Kervadec et al. [2019].

3 Dataset

For this project, we use only the pre-disaster imagery from the xBD dataset xbd [2019], which provides high-resolution satellite images across geographically diverse regions. The pre-disaster subset captures intact building structures under a wide range of environmental conditions, including surface materials, and surrounding context such as roads, parking areas, and tree clusters. Although unaffected by disaster-induced structural changes, these images remain challenging for segmentation due to cluttered urban environments, heterogeneous roof shapes, occlusions from vegetation, and ambiguous boundaries between buildings and background surfaces like concrete, soil, or water.

To train our segmentation model, we converted the vector building annotations in the xBD JSON files into pixel-level binary masks. Each JSON contains building footprints represented as polygons in image coordinates. Our preprocessing script loads these polygons, clips them to the image bounds, and converts them into a 2D binary array where building pixels are assigned 1 and background pixels 0 Alisjahbana et al. [2024]. The resulting masks provide the dense supervision necessary for learning building outlines and distinguishing structures from visually similar background regions.

4 Architecture

We chose a U-Net model because it is a well-established baseline for pixel-wise segmentation and is particularly effective for tasks where both global context and precise spatial localization are required, such as outlining buildings. Its encoder–decoder structure first compresses the image into deep feature representations through successive downsampling (encoder), and then reconstructs a pixel-level prediction map through upsampling (decoder). The skip connections link encoder and decoder features at corresponding resolutions, allowing the model to recover spatial detail lost during downsampling. In our setup, the encoder is a ResNet-34 pretrained on ImageNet, providing strong general-purpose visual features, while the decoder is trained from scratch and fine-tuned on our building segmentation dataset. The final output is a probability map with the same spatial dimensions as the input image, which can be directly overlapped with the ground-truth mask to evaluate performance. We chose the encoder weights of the model pretrained on ImageNet, without too much reasoning, but to get the first experiment started.

For the loss function, we chose to combine Dice loss, selected based on the literature, which measures the overlap between the predicted mask and the ground truth by comparing their intersection relative to their combined size. Dice loss is particularly suitable for our task because buildings occupy only a small portion of most images, and this loss emphasizes correctly capturing the foreground rather than being dominated by the much larger background. We also added binary cross-entropy, since our problem is somewhat a binary classification (building or no building). As for accuracy metric we chose IoU - Intersection over Union, which fits perfectly with our task: IoU is at its highest, if the predicted building segment overlays perfectly with the true polygon and at its lowest if not a single pixel of the predicted building overlaps with the original one. We started with SGD optimizer with no momentum and a learning rate of 0.1, just to get the first experiment going. We will revisit these parameters and how they influence our model later on.

5 Methods and iterations

5.1 Sanity check

The first thing we did was to perform a sanity check to first make sure that we have everything in order and to make sure that what our goals were achievable. We did the overfitting method, which consists on separating from the training dataset a small overfitting set, getting a model to overfit on this small dataset in order to see that our model actually outlines the desired buildings. The sanity check was successful: the model outlined the buildings on this small subset. We could move on this path.

5.2 Dataset reduction and Random crop

This sanity check showed us our first impediment: the training, even for such a small dataset and then the evaluation for the test set was taking too long for our computational powers. We trained our model on a Nvidia RTX 4060 GPU, which helped a lot, but not enough to train on the entire dataset. Because the xBD/xView2 pre-disaster dataset contains

many tiles with little or no building coverage, we thought that randomly reducing the dataset could produce a highly imbalanced distribution dominated by empty scenes. Such an imbalance is known to bias deep models toward majority cases Buda et al. [2018]. To avoid this, we computed the building-area ratio for each image and grouped samples into quantile-based bins representing different building densities. We then performed stratified subsampling within each bin to preserve the density distribution of the dataset. The sampling can be seen in Figure 1. We decided to keep the same ratio because we initially didn't want to lose unique environments like rivers, farmlands, and forests, which could exist in pictures with a lower building ratio, but also because we trusted the reasoning of makers of this challenge when they created this initial training dataset.

Even though we reduced the number of images processed, the training was still was not fast enough to meet our needs. This was because we were keeping the images at their original size 1024x1024, and it was taking a lot of time to load the images. The first thought was to reduce the quality to 512x512, but then we would lose the pixel precision which we need to have exact contours for the buildings. The solution we choose to follow was to perform a random crop on the train images to 256x256 Takahashi et al. [2018], so we preserve the quality and pixel precision. What needs mentioning is that we kept the 1024x1024 size for test images, and this was possible due to the architecture of the model being able to process arbitrary input size. The random crop iteration preserved the IoU for test around 0.55, so we considered it a successful one: we reduced the computational time while preserving the performance.

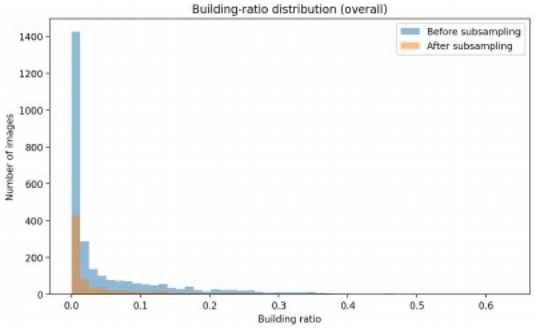


Figure 1: Building ratio for subsampling

5.3 Learning rate

The next problem we identified was that the training loss was steep in the beginning and both the train and test loss and iou curves were spiking a lot from one epoch to another. IoU was hitting 0.57 for next epoch to go at 0.5 and then to rise again after a few epochs and so on, and this at the end part of the training. We deduced that this was because of the high learning rate (high learning rate would make the model jump out from the local minimum and therefore reduce the performance). So we decided to implement a learning rate scheduler to reduce the learning rate, by an initial value of 0.96, over the epochs to help the model find that local minimum. We also applied a learning rate finder to find the best loss to start with. We naively started with the loss equaling 0.1 and after looking at the curve from the loss finder we deducted that the best value lies between 0.01 and 0.005 Figure 2. We got better-looking loss curves, meaning not too steep and pretty continuous, and an iou on test of 0.57, meaning the model was better at learning. We saw that the loss curve usually stabilizes after 50 epochs, so from now on, almost all the experiments would be run for 50 epochs.

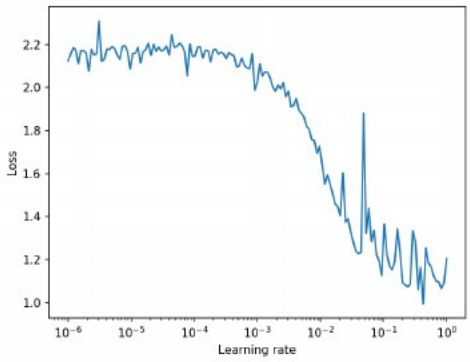


Figure 2: Learning rate finder; steeper = better

5.4 Data normalization

Until this point, the data was not normalized at all. Normalization is essential because unnormalized pixel values shift the data far from the origin, making gradient-based optimization unstable and slowing convergence. Centering and scaling the inputs ensure that all dimensions have comparable magnitude, which stabilizes the loss landscape and leads to more predictable, efficient gradient updates. This directly improves training dynamics, as discussed in the course material on preprocessing. The pixel values are in range [0 - 255]. Firstly, we naively zero-centered the data and we were about to perform mean subtraction when we realized that the data should be normalized with the mean and std from the imagenet, on which the model has been originally pretrained because a pretrained encoder expects the same data distribution it was originally trained on. The encoder we chose was trained on ImageNet images that were zero-centered and normalized per-channel; using the same preprocessing ensures that the early convolutional filters receive inputs within the statistical range they were optimized for. Failing to match this distribution causes internal activations to shift, degrading feature extraction and harming downstream segmentation performance. Unfortunately,

we didn't see any of these to actually happen in our case, since the performance stayed the same. Nevertheless, it is a good practice to normalize the data in this way.

5.5 Vanishing gradients - or not?

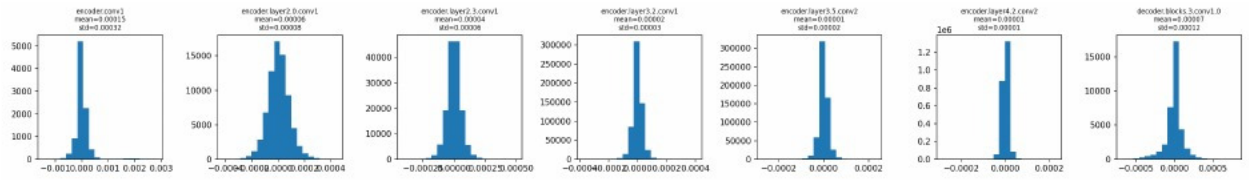
During the sanity check we achieved an IoU over 0.9 while the loss went down around 0.3. Looking at our current the training loss, it was converging around 0.8 so we thought our model was stopping from learning after the initial phase. This could be the cause of high learning rate as shown in Figure 3, problem which we already solved, or vanishing gradients could be another cause.

Figure 3: Loss curve interpretation

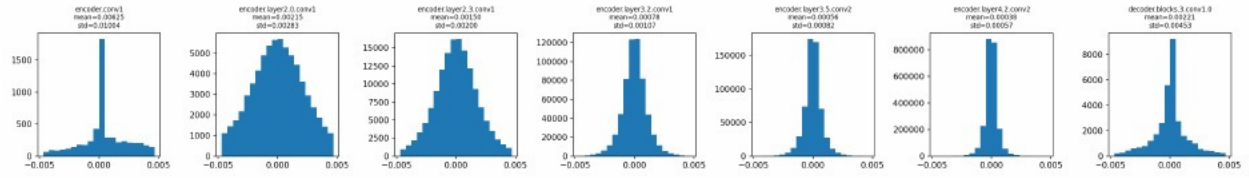
We now realize that this was not the best hypotheses/argumentation, because comparing the model trained on a couple of images vs the model trained on an actual training dataset doesn't really mean anything. But we decided to include this failed experiment as well, because it was a mistake we learned from. We should have realized right away that vanishing gradients could not happen because we had a residual network. Residual networks introduce identity skip-connections that allow gradients to bypass stacked nonlinear layers. This prevents them from shrinking as depth increases, making gradient flow much more stable and eliminating vanishing-gradient problem. Conceptually, residual blocks enable the model to learn residual functions rather than full transformations, which preserves strong gradient propagation even in very deep architectures.

Even more, we checked the initialization of the UNet and it had batch-normalization enabled by default. Meaning that it forces the activations in each layer to have a stable scale and average value during training. This prevents the values inside the network from drifting around too much, makes the model less sensitive to how the weights were initialized, and helps the gradients stay strong so the network can keep learning effectively.

Nevertheless, we decided to see how the gradients were looking. We decided to pick a few equally distant layers and look at the gradients, but one crucial mistake we did was not setting the same scale of x-axis for all layers, but having it computed based on the values of the layers. Therefore, because of the highest and the lowest value, it looked like most of the gradients were squished in the middle bar. After a talk with the lecturer, where he pointed out this mistake, and after we fixed it, we could see that actually there was a lot of gradient flow in our model, Figure 4a and 4b



(a) Wrong visualization of gradients



(b) Correct visuzalization of gradients

Worth mentioning are some small experiments we tried before figuring out that we don't have a vanishing gradients problem. We also experimented with adding momentum to the SGD optimizer. Momentum is designed to smooth the optimization trajectory by accumulating a running average of past gradients. Instead of updating the weights solely based on the current gradient, which may be noisy or point in slightly different directions from batch to batch, momentum keeps part of the previous update and adds it to the new one. This has two important effects: it accelerates learning in directions where gradients consistently point the same way, and it helps the optimizer push through shallow regions of the loss landscape where plain SGD might stall. Since we initially suspected that the model was no longer learning due to overly small or unstable gradients, adding momentum seemed like a reasonable attempt to maintain stronger updates and prevent the optimization from getting stuck. However, the experiments showed no meaningful difference, further confirming that the issue was not related to the optimizer struggling with vanishing gradients in the first place.

We also tried to implement our own residual model, much smaller than the ResNet34, because we thought that the architecture was too big for our problem and the gradients for disappearing due to it, which was not logical because we had a residual network in the first place, but we didn't know in which direction to go exactly. The performance significantly decreased and the gradients looked the same, because of the same scale of the x-axis problem.

5.6 GradCam and Error Heatmap

After this failed experiment, we used Grad-CAM to inspect what the U-Net actually looks at when predicting buildings. Concretely, we hooked into the last convolutional layer of the decoder and, for each test image, backpropagated the mean building logit to obtain a class-activation map. This map was then overlaid on the original RGB image, giving a heatmap of the regions that most strongly influenced the prediction. These visualizations showed that the network does not only respond to rooftops, but frequently also includes nearby objects such as driveways, roads, pools, or other structures around the houses. This suggested that the model was partly solving the task by exploiting correlated background cues instead of isolating the buildings themselves, which helps explain why it struggled to precisely follow roof contours and why further experiments were needed. This can be seen in heatmap 5 how model also looks at driveways and construction sites.

Now that we know that our model was not the best at detecting the margins of the buildings we decided to implement some error heatmaps that will show us more precisely how the model saw the borders of the buildings. This can be seen in Figure 8a how the model outputs a lot of false positives (red coloring) around the smaller buildings.

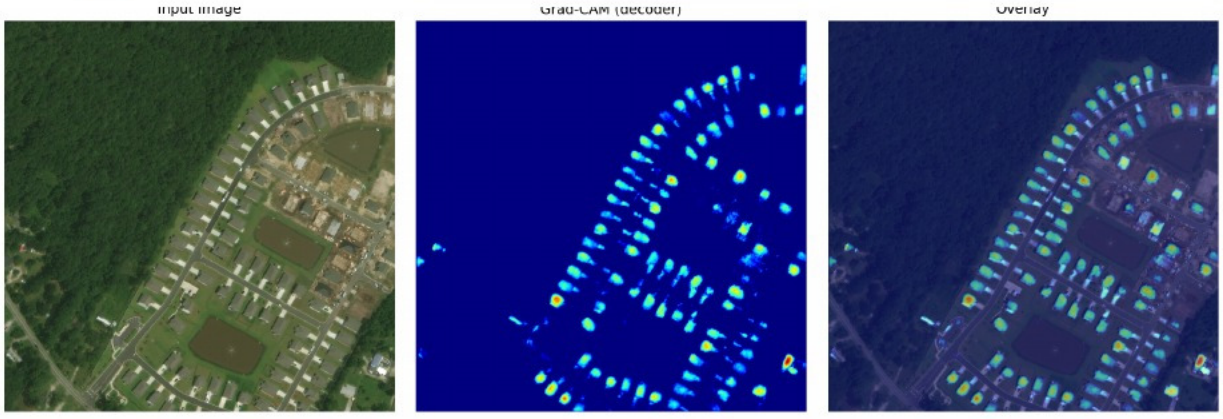


Figure 5: Grad-CAM inspection

5.7 Boundary loss

At first, we trained the model using a loss made of binary cross-entropy and Dice loss. This works well for measuring overall overlap between prediction and ground truth, but it does not pay special attention to the building edges. Since most building pixels are part of large, uniform roof areas, the model can achieve a good score even if it neglects the exact boundary of the building. This was exactly the weakness we observed in our predictions. To improve boundary quality, we introduced a boundary-aware loss. We first extract a thin outline around each building from the ground-truth mask, and then assign higher weights to these edge pixels during training. This could make the model pay more attention to boundary errors, encouraging cleaner, more precise contours instead of including nearby objects like pathways like before. The final loss, therefore, keeps the original BCE + Dice components but adds this boundary weighting, making the model focus not only on detecting buildings but on outlining them. We expected the boundary-aware loss to noticeably improve performance, since it explicitly increases the penalty on errors along building edges. In practice, however, this change had almost no impact. Even after running a separate learning-rate search for this setup, the test IoU remained below 0.6. A likely reason is that the boundary weighting only affects a very thin band of pixels, while the rest of the loss is still dominated by the large interior regions of each building. In addition, our masks and input images are relatively noisy at the pixel level, so moving the focus to single-pixel contours does not necessarily translate into better results.

5.8 Random crop around building - "class" imbalance

Our bottleneck at this point was performance in terms of IoU. In order to increase it, we decided to tweak the way we crop the buildings. Up to this point we have chosen a random crop of the original image, but we saw that the model was

learning very well the areas with just vegetation and little to no buildings. The IoU for the background class was over 0.9. This was also because a "class" imbalance was present in the dataset: a lot of the images contained a very small number of buildings Figure 1.

We decided then to focus more on making the model better at segmenting the buildings, rather then recognizing them from the background. We choose to keep the crop of the images, but we will centre it around a building, to make our model focus on the parts where he was performing worse. However, we did not want to drop all the vegetation images, so we set the rule: half the time it chooses a random crop and half the time it chooses a crop focused on buildings. This change was a step forward in the good direction. We let the model run for only 10 epochs because this method was taking more time than the previous one, but we could see that our train loss was still decreasing, so it needed more epochs to run. Our model managed now at the end of 50 epochs to improve the Iou score on the test set from 0,57 to 0,66. For this run we used a learning rate of 0,01. However we could see that the loss curve was too steep at the start, so we ran a learning rate finder. We found that the drop that we were looking for on the plot was somewhere between 0,001 and 0,01 so we tried first a run with the learning rate set to 0,001 and then one with 0,005. We couldn't see a big difference in the performance, so we went with the learning rate of 0,005 because it had a good-looking loss curve. We also tried to make the model choose more of the crops focused on building, so instead of half the time, we changed it to 90%. This rose the Iou from 0,66 to only 0,67.

5.9 Data augumentation

This experiment was done in parallel with the building-focused cropping. We decided to add some data augmentation to our images, because our model was struggling with the precise margins of the buildings, we thought that some random brightness contrast and color jitter will help it. Our goal here was to change the coloring so the model could learn that a pool for example can be also gray in the image if it was empty, it doesn't necessarily have to be blue, by this we hoped that the model would stop including the pools in the selection for the buildings. However this did not turn out to be as useful as we hoped, our motivation would be that the satellite images were already different enough that this small jittery was not enough for the model.

5.10 Hard negative mining

The last experiment we tried was to implement hard negative sampling. This decision was backed up by the fact that our model, even though it improved, was still not able to define precisely the contour of the houses. We choose this technique to try to force the model to learn better by giving it more difficult training data. Instead of treating all background pixels equally, this method identifies the hardest background examples—the ones the model consistently misclassifies and prioritizes them during training. After computing the loss, we select only the top 25% highest-loss background pixels and use them to update the network. This prevents easy negatives (uniform vegetation or clear open areas) from dominating the loss, and forces the model to repeatedly confront challenging cases such as shadows, pavement, or roof-like textures that often confuse it.

We hoped that directing learning toward these difficult regions, HNM would sharpen the model's discrimination ability and would improve boundary accuracy, making it a valuable technique when the base model has already learned the general shape of buildings but lacks contour precision. However, even though our hypothesis was valid and the heatmap inspection indicated that difficult background pixels were contributing to boundary errors, hard negative mining did not solve the problem. Instead, the model quickly overfitted. The training loss kept decreasing while the test loss highly increased, indicating that the network was learning to fit the difficult pixels in the training set without improving its generalization ability. In other words, HNM intensified the model's focus on the most challenging training negatives, but this specialization did not transfer to unseen data. The technique made the model better at memorizing hard examples from the training distribution rather than learning cleaner decision boundaries.

6 Results

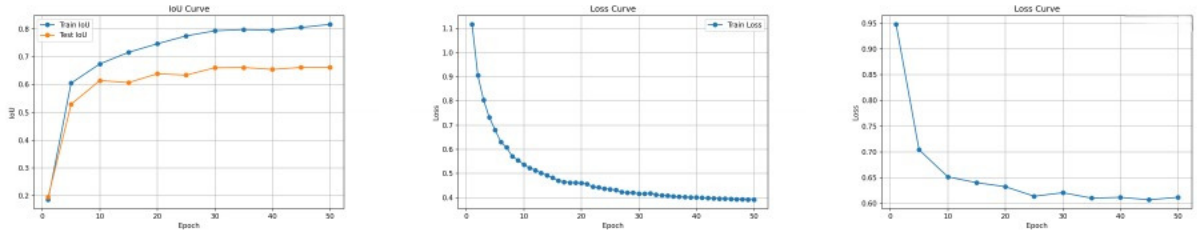
We evaluate our final model and the main intermediate variants using Intersection over Union on the test split. Table 1 summarizes the evolution of performance as we gradually introduced the changes described in the previous section. It also underlines the power of a pretrained model on a suitable architecture.

Our U-Net model, using a ResNet-34 encoder pretrained on ImageNet with a Dice + BCE + boundary loss, with building-centred crops, achieves an IoU of roughly 0.67 for buildings and 0.97 for background on the test set. Almost the same values, 0.66 IoU for buildings and 0.97 for background were the values obtained by initial creators of the xBD dataset xbd [2019]. We can see in Figure 6 nice converging loss curves. There is a small overfit, meaning that the model performs slightly better on the train dataset, but as long as the test loss is decreasing, there is nothing we should worry

Table 1: Overview of main experiments. IoU values are reported on the test split.

Configuration	Crop strategy	Building IoU	Background IoU
Baseline U-Net (SGD, LR 0.1), no normalization	random 256×256	≈ 0.55	≈ 0.93
+ LR scheduler and LR finder	random 256×256	≈ 0.57	≈ 0.96
+ ImageNet-style normalization	random 256×256	≈ 0.57	≈ 0.96
+ boundary-aware loss	random 256×256	≈ 0.57	≈ 0.96
+ data augmentation	random 256×256	≈ 0.57	≈ 0.96
+ building-centered crop (50% of times)	mixed random / centered	≈ 0.66	≈ 0.97
+ stronger focus on building-centered crop (90%)	mostly centered	≈ 0.67	≈ 0.97
+ hard negative mining	mostly centered	worse (overfitting)	worse (overfitting)

about, because the model is still learning to generalize. There is a design mistake, meaning that both loss curves should have been on the same plot, like for IoU, but one can still compare them.



(a) Building IoU curve train and test (b) Train loss curve (c) Test loss curve

Figure 6: IoU and loss curves for the best model

7 Discussion

The sampling strategy had the largest positive impact. Purely random crops emphasize the dominant background class and push the model towards trivial "non-building" predictions, for which IoU exceeds 0.9 on the background but remains much lower on buildings. Introducing building-centered crops, while still keeping a fraction of purely random patches, yielded the jump from ≈ 0.57 to ≈ 0.66 test IoU by forcing the model to focus more on rooftops, cluttered surroundings, and small structures. By contrast, more sophisticated modifications, like loss modification, data augumentation or hard negative mining, did not translate into measurable gains.

(a) Model predicting pools as buildings (b) Model not including pools after cropping around buildings

Figure 7: Moving out pools from False Positive set

We can see this in the prediction of pools as buildings, which was first a motivation for data augumentation, but turned out to be solved, or mostly solved, by doing the random crop around buildings, Figures 7a and 7b. At the same time, we can see in Figures 8a and 8b how the same change of cropping focus from random to building-focused improved

the small building outlines and identification of big buildings, even though it is still struggling with the big buildings' outlines.

(a) Building prediction without building-focused crop

(b) Building prediction without building-focused crop

Figure 8: Error map of model predictions. TP=green, FP=red, FN=blue

8 Conclusion

In this project, we set out to build a reliable model for extracting building footprints from the xBD pre-disaster dataset using a U-Net with a ResNet-34 encoder. Through a series of iterative experiments, we found that the main limitations in our early models did not stem from architecture, loss functions, or optimization choices, but rather from how the data was presented during training. While techniques such as boundary-aware loss, normalization, data augmentation, and hard negative mining offered limited or no improvement, rethinking the sampling strategy had a bigger impact. Ensuring that the model was consistently exposed to informative building regions through building-centered cropping led to the most significant gains.

Grad-CAM visualizations and error heatmaps helped us understand not only where the model succeeded, but also where it systematically misinterpreted contextual cues such as pools, driveways, and other man-made structures. This highlighted that segmentation quality depends as much on the statistical structure of the data as on the model architecture itself. Our final model still struggles with precise outlines of buildings and occasionally produces false positives in visually similar regions, like mistaking big trucks for small buildings, showing that more refined boundary modeling or richer supervision could be beneficial. On crucial aspect that went overlooked was choosing ImageNet for the pretrained encoder weights, while SpaceNet would have been way more suitable for our task.

Overall, the project demonstrates the importance of thoughtful data handling in segmentation tasks with strong spatial imbalance. A relatively standard architecture was sufficient to achieve solid performance once the training distribution was aligned with the task's requirements. Future work could explore more advanced cropping policies, multi-class supervision, or transformer-based encoders to push the boundary of performance further, but the experiments conducted here provide a clear and well-motivated baseline for building segmentation on xBD.

9 Code and Dataset

The code can be found in the github repository: https://github.com/ and the dataset can be downloaded from https://xview2.org/dataset, the first stage's dataset.

References

xview2 challenge. https://xview2.org/. Accessed: 2025-12-04.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In Nassir Navab, Joachim Hornegger, William M. Wells, and Alejandro F. Frangi, editors, *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241, Cham, 2015. Springer International Publishing. ISBN 978-3-319-24574-4.

Waleed Alsabhan and Turky Alotaiby. Automatic building extraction on satellite images using unet and resnet50. *Computational Intelligence and Neuroscience*, 2022(1):5008854, 2022. doi:https://doi.org/10.1155/2022/5008854. URL https://onlinelibrary.wiley.com/doi/abs/10.1155/2022/5008854.

İsa Ataş. Performance evaluation of jaccard-dice coefficient on building segmentation from high resolution satellite images. *Balkan Journal of Electrical and Computer Engineering*, 11(1):100–106, 2023. doi:10.17694/bajece.1212563.

Hoel Kervadec, Jihene Bouchtiba, Christian Desrosiers, Eric Granger, Jose Dolz, and Ismail Ben Ayed. Boundary loss for highly unbalanced segmentation. In M. Jorge Cardoso, Aasa Feragen, Ben Glocker, Ender Konukoglu, Ipek Oguz, Gozde Unal, and Tom Vercauteren, editors, *Proceedings of The 2nd International Conference on Medical Imaging with Deep Learning*, volume 102 of *Proceedings of Machine Learning Research*, pages 285–296. PMLR, 08–10 Jul 2019. URL https://proceedings.mlr.press/v102/kervadec19a.html.

xbd: A dataset for assessing building damage from satellite imagery. 2019. URL https://xview2.org/dataset.

Irene Alisjahbana, Jiawei Li, Yue Zhang, et al. Deepdamagenet: A two-step deep-learning model for multi-disaster building damage segmentation and classification using satellite imagery. *arXiv preprint arXiv:2405.04800*, 2024.

Mateusz Buda, Atsuto Maki, and Maciej A. Mazurowski. A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106:249–259, 2018. ISSN 0893-6080. doi:https://doi.org/10.1016/j.neunet.2018.07.011. URL https://www.sciencedirect.com/science/article/pii/S0893608018302107.

Ryo Takahashi, Takashi Matsubara, and Kuniaki Uehara. Ricap: Random image cropping and patching data augmentation for deep cnns. In Jun Zhu and Ichiro Takeuchi, editors, *Proceedings of The 10th Asian Conference on Machine Learning*, volume 95 of *Proceedings of Machine Learning Research*, pages 786–798. PMLR, 14–16 Nov 2018. URL https://proceedings.mlr.press/v95/takahashi18a.html.